



Instituto Politécnico Nacional

Escuela Superior de Cómputo

Practica 1

Investigación de Bases de Datos y Caso de Estudio

Materia: Bases de Datos

Profesor: Gabriel Hurtado Avilés

Alumno: Damas Garnica Adrian

Grupo: 3CV4

Resumen

En este documento se presenta la documentación incial del desarrollo de una base de datos focalizada en un sistema de comandas.

Índice

1 Introducción **1**
1.1 Planteamiento del problema 1
1.2 Justificación 1
1.3 Objetivo general 2
1.4 Objetivos específicos 2
1.5 Estado del arte 3
1.5.1 Análisis del estado del arte 4
1.6 Productos esperados 5

2 Marco teórico **6**
2.1 Dato, información y bases de datos 6
2.1.1 ¿Qué es un dato? 6
2.1.2 Información 6
2.1.3 Base de datos 6
2.2 Características de una base de datos 7
2.2.1 Integración 7
2.2.2 Persistencia 7
2.2.3 Redundancia controlada 7
2.2.4 Integridad 8
2.2.5 Independencia de datos 8
2.2.6 Seguridad 8
2.2.7 Concurrencia 9
2.2.8 Recuperación 9
2.3 Archivos contra bases de datos 9
2.3.1 Sistemas de archivos 9
2.3.2 Problemas del enfoque en sistemas de archivos tradicionales 10
2.4 Usuarios de una base de datos 10
2.4.1 Administrador de la base de datos 10
2.4.2 Diseñador de la base de datos 11
2.4.3 Programador de aplicaciones 11
2.4.4 Usuario final 12
2.5 Ciclo de vida de una base de datos 12
2.6 Sistema gestor de bases de datos 13
2.6.1 Módulos de componentes principales 13
2.6.2 Lenguajes del SGBD 14
2.6.3 Clasificación de los SGBD 14
2.7 Sistema de bases de datos 15
2.7.1 Arquitectura de tres niveles (ANSI/SPARC) 15
2.7.2 Independencia de datos 15
2.7.3 Arquitectura centralizada 15
2.7.4 Arquitectura cliente-servidor 16
2.8 Modelos de datos 16
2.8.1 Clasificación por nivel de abstracción 16

3 Caso de Estudio **17**
3.1 Entrevista con el Cliente 17
3.1.1 Eventos y mesas 17
3.1.2 Menú y productos 17
3.1.3 Flujo de comandas 18
3.1.4 Cobros y pagos 19
3.1.5 Roles y permisos 20
3.1.6 Reportes 20
3.1.7 Operación y escalabilidad 20
3.2 Requerimientos de la base de datos 21
3.2.1 Entidades y atributos 21
3.2.2 Relaciones 21
3.3 Requerimientos del sistema en general 23
3.4 Diagrama Entidad-Relación 24
3.5 Justificación del modelo 25
3.5.1 Justificación de las entidades 25
3.5.2 Justificación de las relaciones 26
3.5.3 Cómo resuelve el modelo la problemática planteada 27

1 Introducción

En la siguiente sección se abarca la investigación previa para el desarrollo de la base de datos como proyecto final de la materia Bases de Datos, se plantean las bases para el proyecto y el producto esperado de esta investigación.

1.1 Planteamiento del problema

Los salones de eventos que ofrecen servicio de banquetes suelen gestionar la toma y el seguimiento de comandas –los pedidos de alimentos y bebidas realizados por cada mesa o por cada invitado durante un evento– de forma manual, apoyándose en comandas de papel, hojas de cálculo o aplicaciones genéricas que no fueron diseñadas para este giro de negocio. Esta forma de operar genera problemas concretos: la información de los pedidos se duplica o se pierde entre el área de meseros, cocina y caja; no existe un registro histórico confiable de qué se sirvió en cada evento, a qué cliente y a qué costo; el control de existencias de insumos se lleva de manera aproximada, lo que provoca faltantes o compras innecesarias; y la elaboración de la cuenta final de cada evento es lenta y propensa a errores porque depende de conciliar manualmente varias comandas en papel.

Este proyecto retoma y da continuidad a un desarrollo realizado en un semestre anterior, en el que se propuso un primer sistema de comandas de uso general. Sin embargo, aquella versión inicial se apoyaba en un sistema enteramente en C y este unicamente trabajaba con archivos en formato ASCII para almacenar la información, esto causa que dicho sistema tenga relaciones mal definidas, escasa normalización y ausencia de reglas de integridad, lo que dificultaba su mantenimiento y su crecimiento. Por ello, el problema que atiende la presente práctica no es únicamente digitalizar la toma de comandas, sino rediseñar, con una metodología formal de modelado entidad-relación, la base de datos que sustenta dicho sistema, de modo que sea capaz de representar correctamente eventos, clientes, mesas, menús, platillos, insumos, meseros y pagos, garantizando consistencia, integridad y escalabilidad de la información, de esta forma concretar con un sistema profesional y funcional para nuestro proyecto final de la materia de Bases de Datos.

1.2 Justificación

El diseño de una base de datos robusta para un sistema de comandas resulta pertinente porque este tipo de sistema constituye el núcleo operativo de un salón de eventos: de él dependen la asignación de mesas, el control de los pedidos por evento, la disponibilidad de insumos y, en última instancia, la facturación al cliente. Un modelo de datos mal estructurado no solo genera inconsistencias en la información, sino que limita la capacidad del sistema para escalar a medida que el negocio crece o para adaptarse a nuevos requerimientos, como el manejo de menús personalizados, distintos tipos de evento o reportes administrativos.

Desde el punto de vista académico, esta práctica es relevante porque permite aplicar de forma integral los conceptos revisados a la fecha en la materia de Bases de Datos –definición de una base de datos, implementación de una base de datos, modelos entidad relación, uso de docker y uso de PostgreSQL– a un caso de estudio real y ya conocido por el equipo, lo que facilita identificar con claridad los errores de diseño cometidos previamente y corregirlos mediante un modelo entidad-relación formal, con entidades, atributos, relaciones y cardinalidades correctamente definidas.

Desde el punto de vista práctico, contar con una base de datos bien diseñada beneficia directamente la operación de un salón de eventos: reduce el tiempo de atención al reducir errores en la toma de comandas, permite conocer con precisión el costo y las existencias de insumos por evento, y facilita la generación de reportes confiables para la toma de decisiones administrativas, sentando además una base sólida sobre la cual continuar el desarrollo del sistema en etapas posteriores del proyecto.

1.3 Objetivo general

Diseñar el modelo entidad-relación de una base de datos para un sistema de comandas de un salón de eventos, que permita gestionar de manera íntegra y consistente la información de clientes, eventos, mesas, menús, platillos, insumos, meseros y pagos, con el fin de sustentar una versión mejorada del sistema desarrollado previamente.

1.4 Objetivos específicos

1. Investigar y comparar la definición formal de dato, información y base de datos, así como las características fundamentales de un sistema gestor de bases de datos.
2. Identificar y documentar, mediante una entrevista con el cliente, los requerimientos de información y las funciones que debe cumplir el sistema de comandas del salón de eventos.
3. Analizar las deficiencias de diseño presentes en la base de datos del proyecto realizado en el semestre anterior, con el fin de evitarlas en la nueva propuesta.
4. Construir el diagrama entidad-relación del sistema de comandas, incluyendo entidades, atributos, relaciones y cardinalidades.
5. Justificar las decisiones de diseño del modelo propuesto, explicando cómo cada entidad y relación contribuye a resolver la problemática planteada.

1.5 Estado del arte

Referencia	Problema y metodología	Resultados principales	Relación con la Unidad Temática I	Relación con el proyecto
Aguilar Vera et al. (2023)	Los autores realizan una revisión sistemática de la literatura sobre el modelado y la gestión de bases de datos NoSQL. Analizan los procesos de modelado, los diferentes tipos de datos y las herramientas disponibles para su administración.	Se identifican cuatro modelos principales de bases de datos NoSQL: clave-valor, documentos, columnas y grafos. El modelo orientado a documentos destaca por su flexibilidad y uso frecuente.	El articulo se relaciona con la unidad temática 1.4, «Modelos de datos», al ampliar la clasificación vista en clase.	Para el sistema de comandas, permite contrastar el uso de NoSQL con el modelo relacional requerido para manejar cuentas, pagos y comandas.
Taipalus (2024)	Presenta una revisión de 117 estudios que comparan el desempeño de 44 sistemas gestores de bases de datos. También analiza el rigor metodológico y la posibilidad de reproducir los resultados de dichas comparaciones.	El estudio señala que existen pocas comparaciones de alto rigor y reproducibilidad entre SGBD. En los estudios analizados, los sistemas NoSQL presentan, en general, un mejor desempeño que los sistemas NewSQL en operaciones de lectura y escritura.	El articulo se relaciona con la unidad temática 1.2, «Sistema gestor de base de datos», especialmente con su clasificación.	En el proyecto, sirve como referencia para analizar la elección de PostgreSQL considerando no solo el desempeño, sino también las necesidades de transacciones e integridad de los datos.
Machuca & Braul (2022)	Los autores realizan una revisión de 14 artículos publicados entre 2016 y 2021 para identificar las metodologías más utilizadas en la seguridad de bases de datos. Las fuentes consultadas incluyen IEEE, arXiv, Hindawi y Dialnet, tras la revisión se seleccionaron 14 artículos para ser analizados.	El control de acceso es la estrategia de seguridad más utilizada en los trabajos revisados, seguido por técnicas de cifrado y mecanismos de protección para bases de datos en la nube.	Se relaciona con la unidad temática 1.1.1 «Características de una base de datos», en concreto Seguridad e Integridad.	Se relaciona con las características de seguridad e integridad de las bases de datos. Para el sistema de comandas, respalda la implementación de diferentes roles y permisos para usuarios como meseros, personal de cocina, bar y cajeros o administradores.

Tabla 1
Comparación de trabajos relacionados con el proyecto

1.5.1 Análisis del estado del arte

Al revisar los tres trabajos, lo primero que se nota es que ninguno propone un sistema nuevo. Los tres son revisiones sistemáticas de literatura, así que su aporte consiste en reunir y ordenar lo que otros autores ya han publicado para mostrar cómo se encuentra cada área. Aunque tratan temas distintos, parten de una preocupación parecida: las aplicaciones actuales manejan datos cada vez más variados, exigen un mejor desempeño y necesitan proteger la información frente a accesos no autorizados. Estos problemas son los mismos que dan sentido a los contenidos de la Unidad Temática I, lo cual indica que conceptos como modelo de datos o sistema gestor de bases de datos siguen siendo el punto de partida para estudiar las tecnologías actuales.

Cada artículo, sin embargo, mira el problema desde un ángulo diferente. Aguilar Vera et al. (2023) se ocupan del modelado y la clasificación de las bases de datos NoSQL, con lo que amplían lo visto en el apartado 1.4, «Modelos de datos». Taipalus (2024) compara el desempeño de distintos sistemas gestores, tema que corresponde al apartado 1.2, «Sistema gestor de base de datos». Por último, Machuca y Braul (2022) revisan las estrategias de seguridad más empleadas, entre ellas el control de acceso, el cifrado y la protección de bases de datos en la nube, y su trabajo se vincula con las características de seguridad e integridad del apartado 1.1.1. Leídos en conjunto, los tres artículos dejan ver que diseñar una base de datos implica bastante más que elegir un modelo o un gestor; también hay que considerar cómo responderá el sistema ante la carga de trabajo y quién podrá consultar o modificar cada dato.

Otro punto en común es que ninguna de las tres áreas cuenta con criterios del todo estandarizados. En NoSQL conviven varios modelos y diversas formas de modelar. En las comparaciones de desempeño, Taipalus (2024) encuentra pocos estudios rigurosos y reproducibles, además de metodologías que varían mucho entre sí. En seguridad tampoco hay un enfoque único que funcione para todos los escenarios. De aquí se desprende que no existe una tecnología de base de datos superior en términos absolutos y que la elección debe partir de las necesidades de cada sistema.

Esta idea orienta las decisiones de diseño de nuestro proyecto. La revisión de Taipalus (2024) reporta diferencias de desempeño importantes entre familias de gestores, con los sistemas NoSQL por encima de los NewSQL en operaciones de lectura y escritura; sin embargo, en un sistema de comandas la velocidad no es el criterio que más pesa. Una comanda duplicada, un pago mal registrado o una cuenta que no cuadra al cierre de un evento afectan directamente la operación del salón y la confianza del cliente. Por esa razón se da prioridad a la integridad y la consistencia de los datos, y PostgreSQL, al ser un gestor relacional con soporte para transacciones y restricciones de integridad, resulta la opción más adecuada. De manera similar, lo reportado por Machuca y Braul (2022) respalda la definición de roles y permisos diferenciados, de modo que meseros, personal de cocina y bar, cajeros y administradores accedan solo a la información que les corresponde. En resumen, estos trabajos complementan lo estudiado en la Unidad Temática I y, al mismo tiempo, aportan argumentos concretos para sustentar el diseño del sistema.

1.6 Productos esperados

Como resultado de esta práctica se espera obtener un conjunto de entregables documentales y técnicos que evidencien tanto el proceso de investigación como el diseño de la base de datos del sistema de comandas.

Los productos esperados son:

- Descripción del caso de estudio, incluyendo la entrevista con el cliente y la lista de requerimientos del sistema de comandas.
- Diagrama entidad-relación del sistema de comandas para el salón de eventos, con entidades, atributos, relaciones y cardinalidades.
- Las bases de la implementación de la base de datos para el proyecto final.

2 Marco teórico

2.1 Dato, información y bases de datos

2.1.1 ¿Qué es un dato?

De acuerdo con Elmasri y Navathe (2016), un dato «es un hecho que es reconocido y que puede ser registrado con un significado implícito» (p. 4). Un ejemplo de dato podrían ser: un nombre, números de teléfono, ubicaciones, direcciones de correo electrónico, entre muchos más.

Por otro lado, Bellinger et al. (2004) indican que los datos «no tienen un significado aparente más allá de su propia existencia», y estos pueden ser o no utilizables para distintos fines (p. 1).

Por ambas partes los autores nos dan a entender que los datos son trozos de información que se pueden utilizar con distintos fines; Bellinger y sus colaboradores nos dicen que más allá de su existencia no tienen otro significado.

2.1.2 Información

Elmasri y Navathe (2016) señalan que una base de datos se constituye por medio de una serie de datos relacionados entre sí de manera lógica y coherente; por lo tanto, la base de datos genera información con los datos que almacena. De esta forma, no es posible construir una base de datos con una variedad aleatoria de datos sin una conexión entre sí (p. 5).

Bellinger et al. (2004) mencionan que «la información son datos a los que se les ha dado significado a través de una conexión relacional» (p. 1).

Ambos autores nos dicen que los datos tienden a tener una conexión racional y lógica para generar información y que este mismo principio es fundamental y constituye a las bases de datos. Un ejemplo de esto podría ser una calle: «calle 1» por sí sola no tiene sentido, pero si se le relaciona con una persona se puede decir que esa persona vive en la «calle 1».

2.1.3 Base de datos

Para Date (2001), una base de datos «es un sistema computarizado que se utiliza para llevar el registro electrónico de información» (p. 2). Los usuarios y administradores pueden realizar acciones enfocadas en la información que almacenan; por ejemplo, pueden agregar más información, modificar la información existente, recuperarla o eliminar información y datos.

Silberschatz et al. (2002) definen diferencias entre un sistema de procesamiento de datos y una base de datos, indicando que un sistema gestor de bases de datos coordina sistemas físicos y

accesos lógicos, y este mismo garantiza el correcto uso de la información en aplicaciones por medio de consultas (p. 4).

El autor Date nos da una versión más figurada y sencilla de lo que es una base de datos, mientras que Silberschatz y sus colaboradores ofrecen una visión más técnica y detallada sobre las diferencias principales que tiene una base de datos frente a un sistema de procesamiento de datos.

2.2 Características de una base de datos

2.2.1 Integración

Elmasri y Navathe (2016) explican que una base de datos funciona como un único repositorio donde se unifican las estructuras de los datos y la información; de esta forma, reemplaza a los archivos independientes e inconexos (p. 10).

Según Silberschatz et al. (2002), la integración es «la consolidación y la centralización de la información dispersa o descentralizada en múltiples archivos», de forma que permite el acceso unificado a la información (p. 3).

Ambos autores nos dicen la misma idea, donde los archivos independientes o descentralizados se conectan de forma conexa en la base de datos.

2.2.2 Persistencia

De acuerdo con Elmasri y Navathe (2016), los datos almacenados tienen la propiedad de persistir de forma duradera en los medios de almacenamiento no volátiles, sobreviviendo a la finalización o ejecución de los programas que los crearon (p. 20).

Date (2001) afirma que, de forma precisa, los datos de la base de datos «persisten» debido a que una vez son aceptados por el SGBD solo pueden ser removidos por una petición al SGBD, pero no como un efecto lateral si algún programa termina su ejecución (pp. 9–10).

En ambas citas los autores nos plantean ideas similares: que dentro de un SGBD los datos y la información persisten, y esta persistencia siempre estará presente sin importar las ejecuciones que haga la aplicación donde se aloja el SGBD, y solo cambiará por medio de peticiones directas al SGBD.

2.2.3 Redundancia controlada

Elmasri y Navathe (2016) señalan que, aunque la norma es eliminar la duplicación mediante la normalización, cuando es necesario duplicar datos de forma intencional para acelerar las

consultas, el SGBD aplica una redundancia controlada para asegurar que el cambio se haga correctamente por igual entre sus copias (pp. 17–18).

Silberschatz et al. (2002) indican que consiste en evitar la duplicación innecesaria de información (propia de los sistemas de archivos tradicionales) para prevenir inconsistencias; si existe redundancia por diseño, el SGBD mantiene la sincronización (p. 4).

Los autores Elmasri y Navathe profundizan más en el aspecto de cómo funciona dicha duplicidad, y en la otra cita se profundiza en cómo se mantiene una sincronización dentro del SGBD.

2.2.4 Integridad

Elmasri y Navathe (2016) mencionan que la integridad garantiza que los datos cumplan con las restricciones definidas en el catálogo del sistema (como la clave primaria), las cuales el SGBD valida automáticamente en cada actualización (pp. 19–20).

Por su parte, la integridad «asegura la corrección y consistencia de los datos mediante el cumplimiento estricto de reglas de validación como claves primarias especificadas por el administrador» (Silberschatz et al., 2002, p. 5).

En este caso los autores dan la misma idea, donde cierta información específica (describen la clave primaria como ejemplo) tiene que cumplir con la característica de integridad.

2.2.5 Independencia de datos

Según Elmasri y Navathe (2016), es la separación entre los programas de aplicación y los datos gracias a la abstracción. Incluye la independencia física (modificar el almacenamiento sin alterar el esquema conceptual) y la independencia lógica (alterar el esquema conceptual sin modificar las aplicaciones externas) (pp. 12, 36–38).

Silberschatz et al. (2002) la definen como la capacidad de alterar el esquema físico o lógico sin requerir la reescritura ni modificación de los programas de aplicación que acceden a la base de datos (p. 11).

En la primera cita se profundiza más sobre cuáles son las partes del sistema que no interfieren con los datos, ya sea de forma física y/o lógica; la segunda cita es más clara y directa.

2.2.6 Seguridad

Consiste en la protección del sistema contra accesos no autorizados o malintencionados mediante controles de acceso discrecionales (`GRANT`/`REVOKE`), obligatorios (por niveles), cifrado de datos y defensa contra amenazas como la inyección SQL (Elmasri & Navathe, 2016, p. 21).

Silberschatz et al. (2002) la describen como los mecanismos del SGBD que restringen el acceso a la información únicamente a los usuarios autorizados, otorgando privilegios específicos de lectura, modificación o ejecución (p. 5).

Ambos autores nos hablan de cómo el SGBD tiene diferentes mecanismos de seguridad para con el usuario; en la primera se profundiza más en los controles de acceso.

2.2.7 Concurrencia

Elmasri y Navathe (2016) indican que la concurrencia permite que múltiples usuarios y transacciones modifiquen la base de datos simultáneamente. El subsistema de control de concurrencia aplica bloqueos o marcas de tiempo para garantizar la atomicidad y el aislamiento de las operaciones (pp. 13–14, 21).

Silberschatz et al. (2002) señalan que es la supervisión del acceso simultáneo a los datos que evita anomalías o inconsistencias (como lecturas sucias o actualizaciones perdidas), logrando que las ejecuciones concurrentes equivalgan a ejecuciones seriales (p. 5).

En la segunda cita se describen las consecuencias concretas que pueden pasar en caso de una falta de supervisión del sistema. En la primera cita se describe mejor la característica.

2.2.8 Recuperación

Elmasri y Navathe (2016) la definen como la capacidad del SGBD para restaurar la base de datos a un estado consistente tras un fallo de hardware, software o de sistema, utilizando registros en bitácora (*log*), operaciones `UNDO`/`REDO` e hitos de control (p. 21).

Silberschatz et al. (2002) explican que es un conjunto de algoritmos que garantizan las propiedades de atomicidad y durabilidad, deshaciendo los cambios de transacciones no terminadas y asegurando la persistencia de las transacciones confirmadas ante fallos (p. 5).

Ambas citas describen muy bien la característica, pero en la primera se describen las operaciones concretas para ponerlas en práctica.

2.3 Archivos contra bases de datos

2.3.1 Sistemas de archivos

Sistemas de archivos tradicionales: como señalan Elmasri y Navathe (2016, p. 10) y Silberschatz et al. (2002, p. 3), cada aplicación gestiona sus propios archivos de datos de forma independiente, incrustando la estructura de almacenamiento dentro del código fuente de los programas de aplicación.

Sistemas de bases de datos: según Elmasri y Navathe (2016), los datos se almacenan en un repositorio centralizado e integrado. La definición y estructura de los datos no reside en las aplicaciones, sino en un **catálogo o diccionario de datos (metadatos)** administrado por el SGBD (pp. 11–12).

2.3.2 Problemas del enfoque en sistemas de archivos tradicionales

1. **Redundancia de datos:** Elmasri y Navathe (2016, p. 17) y Silberschatz et al. (2002, p. 4) indican que la falta de planificación centralizada ocasiona que la misma información o datos se almacenen repetidamente, y eso ocasiona desperdicio de espacio en unidades físicas.

2. **Inconsistencia de datos (datos duplicados y contradictorios):** Silberschatz et al. (2002) mencionan que al modificar un dato en un archivo pero omitir la actualización en otro, el sistema almacena información contradictoria para la misma entidad, destruyendo la confiabilidad del sistema y volviendo a caer en la redundancia de datos (p. 4).

3. **Dependencia programa-datos:** de acuerdo con Elmasri y Navathe (2016, p. 11) y Silberschatz et al. (2002, p. 4), si la estructura física y el formato de las declaraciones de archivos están codificados dentro de cada programa, cualquier cambio en la estructura de un archivo exige modificar, recompilar y probar todos los programas que acceden a él.

4. **Dificultad de acceso:** Silberschatz et al. (2002) explican que los sistemas de archivos solo permiten consultas predeterminadas mediante programas compilados. No existen lenguajes de consulta como el lenguaje SQL, por lo que obtener un reporte o siquiera alguna consulta requiere codificar un nuevo programa desde cero (p. 4).

2.4 Usuarios de una base de datos

En un entorno de bases de datos coexisten una variedad de usuarios y perfiles técnicos y operativos involucrados en diferentes áreas de uso en las bases de datos.

Según Elmasri y Navathe (2016), en los sistemas corporativos «muchas personas participan en el diseño, uso y mantenimiento de una base de datos grande con cientos o miles de usuarios», clasificándolos primordialmente como los «actores en el escenario» (p. 15).

2.4.1 Administrador de la base de datos

Responsabilidades: el administrador de la base de datos tiene la máxima autoridad técnica y operativa en el entorno. Según Elmasri y Navathe (2016), «en un entorno de base de datos, el recurso primario es la propia base de datos y el recurso secundario es el DBMS y el soft-

ware asociado; administrar estos recursos es responsabilidad del administrador de la base de datos» (p. 15). Date (2001) también nos dice que el «administrador de datos es un profesional IT. El trabajo del DBA consiste en crear la base de datos real e implementar los controles técnicos necesarios para hacer cumplir las diversas decisiones de las políticas hechas por el administrador de datos» (p. 16).

Permisos: el DBA tiene el nivel más alto de autorización dentro del entorno (privilegios de administrador). Como explican Elmasri y Navathe (2016), «la mayoría de los sistemas de base de datos contienen comandos privilegiados que solo pueden ser utilizados por el personal del DBA» (p. 15).

2.4.2 Diseñador de la base de datos

Responsabilidades: el diseñador se encarga de determinar el modelo lógico de la implementación del sistema y el uso de la información de este mismo. En palabras de Elmasri y Navathe (2016), «los diseñadores de bases de datos son responsables de identificar los datos que se almacenarán en la base de datos y de elegir las estructuras apropiadas para representar y almacenar estos datos», y detallan que los diseñadores deben «comunicarse con todos los posibles usuarios de la base de datos para comprender sus requisitos y crear un diseño que satisfaga estas necesidades» (p. 15).

Permisos: los permisos que tienen se centran en el modelado estructural del entorno, desarrollo y *testing*; posee permisos para modificar la estructura lógica de las bases de datos, tablas e índices.

2.4.3 Programador de aplicaciones

Responsabilidades: este rol tiene la responsabilidad de hacer funcionar el SGBD dentro de una aplicación por medio de código y herramientas de desarrollo. Según exponen Elmasri y Navathe (2016), «los analistas de sistemas determinan los requisitos de los usuarios finales y desarrollan especificaciones para transacciones programadas estándar, mientras que los programadores de aplicaciones implementan estas especificaciones como programas; luego prueban, depuran, documentan y mantienen estas transacciones enlatadas» (p. 16).

Permisos: los desarrolladores tienen permisos de manipulación de datos sobre los esquemas de desarrollo y *testing*. Elmasri y Navathe (2016) señalan que su labor exige interactuar con precompiladores que extraen comandos DML incrustados en lenguajes anfitriones, por lo que disponen de privilegios para ejecutar operaciones como `SELECT`, `INSERT`, `UPDATE` y `DELETE`, además de compilar procedimientos almacenados y disparadores (*triggers*) (p. 43).

2.4.4 Usuario final

Responsabilidades: los usuarios finales son los destinatarios directos de los SGBD, aquellos a los que va dirigido el uso del sistema. Elmasri y Navathe (2016) dicen que «los usuarios finales son las personas cuyos trabajos requieren acceso a la base de datos para consultar, actualizar y generar informes; la base de datos existe principalmente para su uso» (p. 15).

Permisos: los permisos para los usuarios finales se rigen por el principio de mínimo privilegio, donde solo tienen permitidos ciertos permisos según sea el caso. Según Elmasri y Navathe (2016), «a los usuarios o grupos de usuarios se les asignan números de cuenta protegidos por contraseñas. Algunos usuarios solo pueden tener permiso para recuperar datos, mientras que a otros se les permite recuperar y actualizar» (p. 19).

2.5 Ciclo de vida de una base de datos

Como explican Elmasri y Navathe (2016), una base de datos real «posee un ciclo de vida de muchos años, por lo que el DBMS debe ser capaz de mantener el sistema de base de datos permitiendo que evolucione conforme los requerimientos cambian a lo largo del tiempo» (p. 8). Esto se realiza con el fin de evitar que el proyecto fracase o genere inconsistencias. Chen (1976) plantea que el desarrollo debe dividirse en etapas progresivas que van desde la idea en el negocio hasta la puesta en marcha técnica, produciendo cada una de ellas un resultado hasta llegar al final (pp. 9–11).

Planificación del proyecto: antes de modelar, el equipo evalúa si el proyecto tiene sentido práctico y financiero. Se identifican los problemas actuales (como archivos duplicados o procesos manuales lentos), se define el alcance, los costos previstos y si la infraestructura actual soportará la solución. El entregable sería un documento de estudio de viabilidad y plan del proyecto.

Recolección y análisis de requerimientos: esta etapa consiste en entender qué necesita la organización o proyecto. De acuerdo con Elmasri y Navathe (2016), «el diseño de una base de datos completamente nueva comienza con una fase denominada especificación y análisis de requerimientos. Estos requisitos se documentan en detalle para que puedan mantenerse, modificarse y transformarse fácilmente en una implementación» (p. 8). El documento entregable para esta etapa sería un reporte de requerimientos específicos.

Diseño conceptual: una vez que se tienen los requerimientos se crea un modelo semántico que describe el diseño conceptual de la base de datos. Chen (1976) fundamentó este paso demostrando que representar el mundo real mediante entidades y relaciones permite organizar la información de forma natural y sin sesgos tecnológicos (pp. 9–10). El entregable para esta etapa es un esquema conceptual donde se hagan diagramas entidad-relación.

Diseño lógico: Elmasri y Navathe (2016) explican que esta etapa es el puente entre el modelo conceptual y el paradigma del gestor elegido, usualmente el relacional (p. 62). Siguiendo las reglas de derivación introducidas por Chen (1976, p. 28) y formalizadas por Elmasri y Navathe

(2016, p. 289), el entregable es el esquema lógico relacional donde se presentan tablas con las definiciones de llaves e integridad referencial.

Diseño físico: de acuerdo con Date (2001, p. 43) y Elmasri y Navathe (2016, p. 37), es la adaptación del diseño lógico al almacenamiento real en disco o memoria secundaria. Como indican Elmasri y Navathe (2016), el objetivo es «estructurar los datos en el almacenamiento garantizando un buen rendimiento en las consultas y transacciones más frecuentes» (p. 643). El entregable es un esquema físico e interno donde se emplea código SQL y sentencias DDL con parámetros definidos.

Implementación: Date (2001, p. 41) y Elmasri y Navathe (2016, p. 31) señalan que en esta etapa el diseño cobra vida en el motor, compilando el DDL para crear las tablas vacías en el DBMS y pasando al estado inicial cargando los datos reales o migrados. Asimismo, Elmasri y Navathe (2016) explican que a la par se enlazan las aplicaciones cliente (mediante *drivers* como JDBC u ODBC) y se somete el sistema a pruebas de concurrencia y seguridad (pp. 43–45). El entregable es la base de datos en producción junto con sus bitácoras operativas y reportes.

Operación: Date (2001, p. 42) y Elmasri y Navathe (2016, pp. 15, 45) describen esta fase como la vida cotidiana del sistema en producción. Los usuarios finales operan las aplicaciones y el administrador (DBA) supervisa que todo marche con fluidez: aplica respaldos periódicos, ajusta índices si las consultas se vuelven lentas y modifica estructuras cuando el negocio crece o cambia. El entregable en muchos casos son los informes de mantenimiento de la base de datos, entre otros.

2.6 Sistema gestor de bases de datos

Un Sistema Gestor de Bases de Datos (DBMS) es el conjunto de software que media entre los archivos físicos en disco, los usuarios y las aplicaciones clientes. Como señalan Elmasri y Navathe (2016), «el DBMS es un sistema computarizado que facilita los procesos de definir, construir, manipular y compartir bases de datos entre diversos usuarios y programas de aplicación» (p. 6).

2.6.1 Módulos de componentes principales

De acuerdo con la arquitectura descrita por Elmasri y Navathe (2016, pp. 41–44) y Silberschatz et al. (2002, pp. 4–5), el motor se organiza en el procesador de consultas y el gestor de almacenamiento, los cuales se conforman por los siguientes módulos:

Compilador de DDL: interpreta las sentencias de definición de esquemas y vuelca las definiciones y restricciones semánticas directamente en el catálogo del sistema en forma de metadatos.

Compilador de DML y precompilador: el precompilador extrae las instrucciones DML incrus-

tadas en código anfitrión, mientras que el compilador DML las traduce a llamadas de bajo nivel comprensibles por el procesador de ejecución.

Optimizador de consultas: examina las distintas formas de resolver una consulta declarativa y elige el plan de ejecución con menor costo estimado de CPU y operaciones de entrada/salida (E/S). Según explican Elmasri y Navathe (2016), «la optimización de consultas es la actividad realizada por un optimizador de consultas en un DBMS para seleccionar la mejor estrategia disponible para ejecutar la consulta» (p. 691).

Procesador de base de datos en tiempo de ejecución: es el motor central que recibe las consultas compiladas y las transacciones parametrizadas, verifica los permisos y esquemas con el catálogo y comanda la ejecución de las operaciones.

2.6.2 Lenguajes del SGBD

En los sistemas actuales basados en SQL, las instrucciones conviven en una interfaz integrada, pero conceptualmente corresponden a cuatro sublenguajes con objetivos distintos (Elmasri & Navathe, 2016, pp. 38–41; Silberschatz et al., 2002, pp. 42–44, 74).

Lenguaje de Definición de Datos (DDL): utilizado por los diseñadores y el DBA para especificar el esquema conceptual, crear tablas, índices, dominios y reglas de integridad estructural (Elmasri & Navathe, 2016, p. 39).

Lenguaje de Manipulación de Datos (DML): permite a los programas y usuarios interactivos buscar, insertar, modificar o borrar registros. Como puntualizan Elmasri y Navathe (2016), «los DML de alto nivel, como SQL, pueden especificar y recuperar muchos registros en una sola sentencia; por lo tanto, se denominan orientados a conjuntos (*set-oriented*) y declarativos» (p. 40).

Lenguaje de Control de Datos (DCL): proporciona las primitivas de seguridad para gestionar los derechos de acceso de las cuentas de usuario sobre los objetos de la base de datos (Elmasri & Navathe, 2016, p. 19; Silberschatz et al., 2002, p. 74).

Lenguaje de Control de Transacciones (TCL): delimita las unidades atómicas de trabajo garantizando que las modificaciones se apliquen de forma permanente o se descarten por completo si ocurre un error (Silberschatz et al., 2002, pp. 175–176).

2.6.3 Clasificación de los SGBD

Relacionales (RDBMS / SQL): organizan la información en tablas o relaciones bidimensionales regidas por llaves y álgebra relacional, como PostgreSQL, MySQL, Oracle, etc. Chen (1976) demostró que estos modelos derivan limpiamente de las entidades y relaciones del mundo real (pp. 9–11).

NoSQL y almacenamiento Big Data: diseñados para escalabilidad horizontal masiva y es-

quemas flexibles o semiestructurados (Elmasri & Navathe, 2016, pp. 51–52).

2.7 Sistema de bases de datos

Para evitar que los programas de aplicación dependan directamente de la forma en que los datos se almacenan físicamente en el disco, los sistemas de bases de datos modernos separan las aplicaciones de la estructura física subyacente.

2.7.1 Arquitectura de tres niveles (ANSI/SPARC)

Propuesta por el comité ANSI/SPARC, esta arquitectura busca aislar las aplicaciones de usuario de los detalles físicos del almacenamiento (Elmasri & Navathe, 2016, p. 36). Como señalan Elmasri y Navathe (2016), «el objetivo de la arquitectura de tres esquemas es separar las aplicaciones de usuario de la base de datos física» (p. 37).

Nivel interno: posee un esquema interno que describe la estructura de almacenamiento físico de la base de datos (archivos, métodos de acceso, índices, compresión y ordenamiento de registros) (Elmasri & Navathe, 2016, p. 37).

Nivel conceptual: describe la estructura lógica global de toda la base de datos para la comunidad de usuarios, ocultando los detalles de almacenamiento (Elmasri & Navathe, 2016, p. 37).

Nivel externo: comprende diversos esquemas externos o vistas de usuario. Cada vista describe únicamente la porción de datos de interés para un grupo específico de usuarios, ocultando el resto de la base de datos por razones de simplicidad o seguridad (Elmasri & Navathe, 2016, p. 37).

2.7.2 Independencia de datos

La independencia de datos es la capacidad de modificar el esquema en un nivel del sistema sin tener que alterar el esquema del nivel superior ni los programas de aplicación (Elmasri & Navathe, 2016, pp. 37–38; Silberschatz et al., 2002, p. 4).

2.7.3 Arquitectura centralizada

En este esquema, toda la funcionalidad del sistema se ejecuta en una sola máquina: el procesamiento del SGBD, la ejecución de los programas de aplicación y la lógica de la interfaz de usuario (Elmasri & Navathe, 2016, pp. 46–47). Los usuarios se conectaban originalmente mediante terminales sin capacidad de cómputo que solo mostraban texto e interfaces de pantalla (Elmasri & Navathe, 2016, p. 46).

2.7.4 Arquitectura cliente-servidor

Aprovecha la capacidad de cómputo en el extremo del usuario (PC, estaciones de trabajo o dispositivos móviles), dividiendo las cargas de trabajo entre dos o más capas lógicas (Elmasri & Navathe, 2016, p. 47).

2.8 Modelos de datos

Un modelo de datos es una colección integrada de conceptos destinados a describir la estructura de una base de datos, incluyendo los tipos de datos, sus relaciones, restricciones semánticas y el conjunto de operaciones básicas de manipulación y consulta. Como señalan Elmasri y Navathe (2016), «una característica fundamental del enfoque de bases de datos es que proporciona cierto nivel de abstracción de datos. Un modelo de datos, como una colección de conceptos que pueden utilizarse para describir la estructura de una base de datos, proporciona los medios necesarios para lograr esta abstracción» (p. 32).

2.8.1 Clasificación por nivel de abstracción

De acuerdo con Elmasri y Navathe (2016, pp. 33–34), los modelos se ordenan en tres categorías según la cercanía que guardan con la percepción humana frente a los dispositivos de almacenamiento:

Modelos de datos conceptuales: emplean conceptos intuitivos muy cercanos a la percepción de los usuarios finales y alejados de los detalles de hardware (Elmasri & Navathe, 2016, p. 33). Sus elementos fundamentales son las entidades (objetos del mundo real), los atributos (propiedades descriptivas) y las relaciones (asociaciones entre entidades) (Chen, 1976, p. 11; Elmasri & Navathe, 2016, p. 33). El ejemplo canónico es el modelo Entidad-Relación y su extensión EER (Elmasri & Navathe, 2016, p. 33).

Modelos de datos de representación o implementación: se sitúan en un punto intermedio: son comprensibles para los usuarios finales, pero se corresponden directamente con las estructuras organizativas que implementan los motores comerciales. Ocultan los detalles de almacenamiento en sectores de disco, pero representan los datos mediante estructuras de registros o tablas (Elmasri & Navathe, 2016, p. 33).

Modelos de datos físicos: describen cómo se almacenan físicamente los datos en los medios secundarios (discos magnéticos o unidades de estado sólido) (Elmasri & Navathe, 2016, p. 33). Como especifican Elmasri y Navathe (2016), «los modelos de datos físicos describen cómo se almacenan los datos como archivos en la computadora representando información como formatos de registro, ordenamiento de registros y rutas de acceso» (p. 34).

3 Caso de Estudio

En esta sección se abarca la problemática a trabajar a lo largo del semestre para culminarlo en el proyecto final de la materia de Bases de Datos. La problemática que abarcaremos será un sistema de comandas, siendo nuestro primer cliente un salón de eventos enfocándose en el control del flujo de trabajo y el inventario de la mercancía durante los evento, ademas de mantener controlado pagos y estados de pedidos; por lo tanto, la primera instancia del diseño de nuestro sistema estará enfocada en atender las necesidades de este primer cliente.

Además, se ha de aclarar que este proyecto ya tiene un sistema inicial de apoyo, siendo una *interfaz de usuario en terminal* (TUI) hecha con `ncurses`, la cual se desarrolló en la materia de Sistemas Operativos.

3.1 Entrevista con el Cliente

3.1.1 Eventos y mesas

- ¿El salón atiende uno o varios eventos simultáneamente?

  R: Se suele atender un único evento a la vez.

- ¿Cuántas mesas suelen usar por evento?

  R: El número varía, pero suelen ser alrededor de 20 mesas.

- ¿Esa cantidad varía entre eventos?

  R: Sí.

- ¿Las mesas están numeradas?

  R: Sí, cada mesa tiene su número.

- ¿Todos los meseros atienden todas las mesas o se le asigna a cada mesero un número de mesas?

  R: Dependiendo del evento; a veces solo hay un mesero, pero de ser múltiples se asigna a cada mesero un grupo de mesas.

3.1.2 Menú y productos

- ¿El menú es un todo incluido por evento o se atiende a la carta?

  R: La carta es aparte; en los eventos se puede pedir comida preparada con antela-

ción, pero siempre está la opción de la carta.

- ¿Qué categorías de productos manejan?

  R: Alimentos, bebidas con y sin alcohol, y dulcería.

- ¿Los productos tienen precio fijo?

  R: Durante el evento sí, pero tienen tendencia a cambiar por temporada.

- ¿Se manejan variantes de platillos?

  R: Sí, algunas bebidas o platillos tienen tamaños distintos o preparaciones distintas.

- ¿Se necesitan registrar ingredientes o alérgenos por producto, o solo nombre y precio?

  R: Nombre, precio e ingredientes.

- ¿Les sirve una descripción corta de cada producto para mostrar en la carta?

  R: Sí, una descripción breve de qué lleva o cómo se prepara.

- ¿Llevan control del inventario de ingredientes? ¿Necesitan saber cuánto se gasta de cada uno por platillo?

  R: Sí, cada platillo o bebida tiene su receta con las cantidades, y queremos saber cuánto queda de cada ingrediente y cuándo hay que resurtir.

- ¿Un mismo ingrediente se usa en varios productos?

  R: Sí, por ejemplo el tomate o el hielo se usan en muchos platillos y bebidas.

- ¿El menú cambia por temporada o es fijo todo el año?

  R: Tenemos un menú fijo.

3.1.3 Flujo de comandas

- ¿Cómo levantan hoy en día una comanda? (papel, apps sueltas, ninguno)

  R: Los meseros usan comandas de papel y las entregan con el responsable en caja.

- ¿Quién toma la orden: el mesero en la mesa o se centraliza en un punto?

  R: Se centraliza en la barra, donde está el responsable de caja.

- ¿La comanda va directo a cocina o bar, o pasa por un supervisor?

  R: Pasa por el responsable en caja.

- ¿Se necesita un estado para la comida? (pendiente, en preparación, lista, entregada, cancelada)

  R: Sí.

- ¿Puede una mesa hacer varias rondas de pedidos durante un evento?

  R: Sí, un cliente puede pedir varias veces durante un evento.

- ¿Se registran notas especiales por pedido?

  R: De ser necesario, sí.

- ¿Quién puede cancelar o modificar una comanda ya enviada a cocina o bar?

  R: Encargado de caja.

3.1.4 Cobros y pagos

- ¿El cobro se hace al organizador del evento en una sola cuenta, o se divide por mesa, por invitado o por grupo de invitados?

  R: Se puede manejar tanto como un solo cobro al organizador, un cobro por mesa, por invitado o por grupo de invitados. Aunque lo más común es cobrar por grupo de invitados, de los cuales suele haber 2 o 3 por mesa (2 a 3 cuentas por mesa).

- ¿Manejan anticipos o depósitos antes del evento?

  R: Sí.

- ¿Qué formas de pago se aceptan y necesitan quedar registradas?

  R: Se aceptan pagos en efectivo, transferencia o tarjeta.

- ¿Se requiere factura fiscal (CFDI) o solo un recibo interno?

  R: No.

- ¿Hay productos incluidos en el paquete que no se cobren aparte?

  R: Únicamente la comida preparada con antelación.

- De ser así, ¿el consumo extra sí se cobra?

  R: Sí, el consumo extra a la carta se cobra.

3.1.5 Roles y permisos

- ¿Qué roles existen y qué debe poder hacer cada uno en el sistema? (mesero, cocina, bar, cajero, administrador)

  R: Hay mesero, cocina, bar y cajero/administrador.

- ¿Se necesita saber qué mesero levantó cada comanda?

  R: Sí, es necesario saber qué mesero hizo qué orden y qué cuentas lleva.

3.1.6 Reportes

- ¿Qué reportes necesita el negocio? (ventas por evento, estadísticas por producto, tiempo promedio de preparación, ingresos por periodo)

  R: Serían de utilidad, aunque no necesarios, las ventas por evento y los ingresos por evento.

- ¿Necesita comparar el consumo real contra lo presupuestado en el paquete del evento?

  R: Sí.

- ¿Con qué frecuencia se consultan estos reportes? (durante el evento en vivo o solo en el cierre)

  R: En su mayoría en el cierre, pero también durante el evento en vivo.

3.1.7 Operación y escalabilidad

- ¿Cuántos eventos atienden en promedio al mes y cuántas comandas por evento?

  R: Alrededor de quince eventos mensuales, y en un evento normal suelen ser entre 1 y 3 pedidos por cuenta, pero esto se puede llegar a extender.

- ¿El sistema debe funcionar sin internet dentro del salón, o siempre tienen conexión?

  R: Preferentemente que funcionase sin internet.

- ¿Planean usar el sistema en otra sucursal a futuro?

  R: No por el momento.

3.2 Requerimientos de la base de datos

3.2.1 Entidades y atributos

- **Evento**: organizador/cliente, fecha, tipo de evento, número de mesas, paquete contratado, estado del evento.
- **Mesa**: número de mesa.
- **Cuenta**: identificador, estado (abierta/cerrada), total acumulado.
- **Mesero** / **Usuario**: nombre, rol (mesero, cocina, bar, cajero/administrador).
- **Producto**: nombre, descripción, categoría (alimentos, bebidas con alcohol, bebidas sin alcohol, dulcería), precio, variantes (tamaño o preparación distinta).
- **Ingrediente**: nombre, unidad de medida, stock actual, stock mínimo.
- **Comanda** (pedido): fecha/hora, estado (pendiente, en preparación, lista, entregada, cancelada), detalles.
- **Pago**: monto, forma de pago (efectivo, transferencia, tarjeta), fecha.
- **Anticipo**: monto, fecha.
- **Paquete del evento**: monto presupuestado, productos incluidos.

3.2.2 Relaciones

- **Mesa — Mesero/Usuario**: un mesero atiende una o varias mesas; una mesa puede ser atendida por más de un mesero.
- **Mesa — Cuenta**: una mesa puede tener varias cuentas (grupos de invitados).
- **Cuenta — Comanda**: una cuenta genera varias comandas (rondas) durante el evento.
- **Mesero/Usuario — Comanda**: un mesero levanta varias comandas; cada comanda la levanta un solo mesero, pero más usuarios la atienden.
- **Comanda — Producto**: cada comanda hace referencia a un producto.
- **Producto — Ingrediente** (receta): un producto lleva uno o varios ingredientes y un ingrediente se usa en varios productos; de cada par se guarda la cantidad que lleva una porción.
- **Cuenta — Pago**: una cuenta puede recibir uno o varios pagos.
- **Evento — Anticipo**: un evento puede tener uno o varios anticipos.

- **Paquete del evento — Evento**: un paquete puede estar en varios eventos y cada evento puede tener uno o varios paquetes.
- **Paquete del evento — Producto**: el paquete incluye uno o varios productos.

3.3 Requerimientos del sistema en general

- Registrar un evento junto con sus mesas y asignar mesero(s) a cada una (un solo mesero si el evento es pequeño, varios repartidos por grupos de mesas si es grande).
- Permitir que una misma mesa tenga varias cuentas independientes (grupos de invitados), cada una con su propio consumo y cobro.
- Centralizar la toma de pedidos en caja: el cajero recibe la comanda (hoy en papel) y la envía a cocina/bar.
- Actualizar el estado de cada comanda (pendiente → en preparación → lista → entregada) y permitir su cancelación o modificación aun después de enviada.
- Permitir múltiples rondas de pedidos por cuenta durante el mismo evento.
- Registrar notas especiales por producto dentro de un pedido cuando sea necesario.
- Registrar la receta de cada producto (qué ingredientes lleva y en qué cantidad) y descontar del stock de ingredientes lo que se consume al preparar cada comanda.
- Avisar cuando un ingrediente baje de su stock mínimo y marcar como no disponibles los productos que ya no se pueden preparar.
- Registrar qué mesero levantó cada comanda y qué cuentas lleva, para control de responsabilidad y propinas.
- Diferenciar accesos por rol: mesero, cocina, bar y cajero/administrador, cada uno con las funciones que le corresponden.
- Registrar anticipos/depósitos del evento antes de la fecha, y aplicarlos al cobro final.
- Registrar pagos por cuenta con su forma de pago (efectivo, transferencia o tarjeta); no se requiere generar factura fiscal (CFDI), solo un recibo interno.
- Distinguir entre consumo incluido en el paquete del evento (comida preparada con antelación, no se cobra aparte) y consumo extra a la carta (sí se cobra).
- Generar reportes de ventas e ingresos por evento.
- Generar un reporte comparativo entre el consumo real y lo presupuestado en el paquete del evento.
- Permitir consultar los reportes tanto en vivo durante el evento como al cierre.
- Operar preferentemente sin conexión a internet dentro del salón.
- Soportar el volumen operativo actual: ∼15 eventos al mes, hasta ∼20 mesas por evento, 2–3 cuentas por mesa y de 1 a 3 pedidos por cuenta (con posibilidad de más rondas).

3.4 Diagrama Entidad-Relación

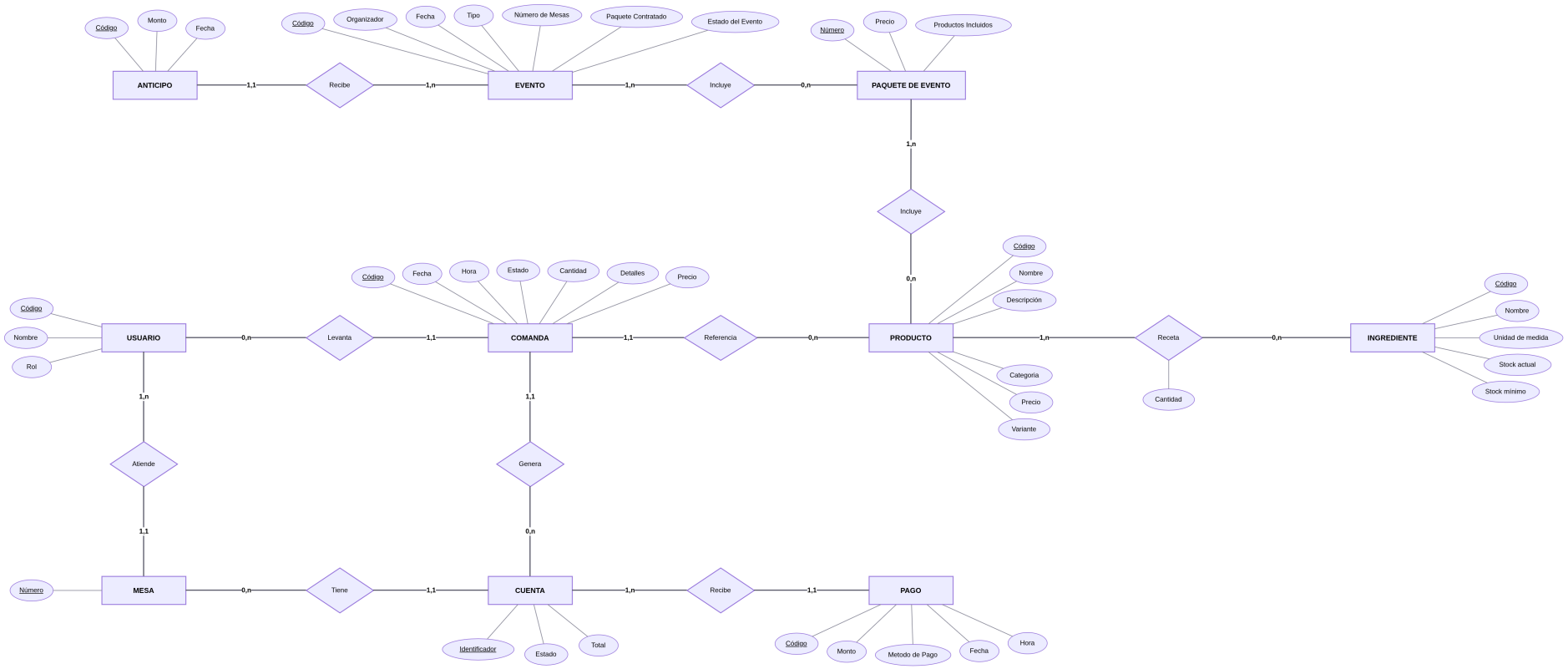


Figura 1: Diagrama entidad-relación

3.5 Justificación del modelo

El modelo entidad-relación de la Figura 1 se construyó a partir de las respuestas obtenidas en la entrevista con el cliente y de los requerimientos que se derivaron de ella. Cada entidad representa un objeto del negocio sobre el cual el salón necesita guardar información de forma permanente, y cada relación representa una regla de operación que el cliente describió. En esta sección se explica por qué se definieron así y de qué manera el diagrama atiende la problemática planteada.

3.5.1 Justificación de las entidades

- **Evento**: es el punto de partida de la operación del salón, ya que el cliente atiende un único evento a la vez y alrededor de quince al mes. Se modela como entidad porque cada evento tiene datos propios (organizador, fecha, tipo, número de mesas y estado) y porque a él se asocian los anticipos y los paquetes contratados. El atributo *estado* permite distinguir un evento programado de uno en curso o ya cerrado.

- **Mesa**: el cliente indicó que las mesas están numeradas y que su cantidad varía entre eventos. Se define como entidad, y no como un simple atributo, porque sobre ella recaen dos reglas del negocio: la asignación de meseros y la división de la cuenta en grupos de invitados.

- **Cuenta**: surge de la respuesta más relevante sobre cobros: lo más común es cobrar por grupo de invitados, con dos o tres cuentas por mesa. Separar la cuenta de la mesa permite que cada grupo tenga su propio consumo, su propio *total* y su propio *estado* (abierta o cerrada), sin mezclarse con el de otros grupos sentados en la misma mesa.

- **Usuario**: agrupa en una sola entidad a todas las personas que operan el sistema. En lugar de crear una entidad distinta para meseros, cocina, bar y caja, se utiliza el atributo *rol*, ya que todos comparten los mismos datos (código y nombre) y solo difieren en las funciones que pueden realizar. Esto simplifica el modelo y facilita agregar roles en el futuro.

- **Producto**: representa los artículos del menú, que el cliente clasificó en alimentos, bebidas con y sin alcohol, y dulcería. Los atributos *categoría* y *variante* responden a que algunos platillos y bebidas se ofrecen en distintos tamaños o preparaciones. Como el menú es fijo durante todo el año, el producto funciona como catálogo estable al que hacen referencia las comandas y los paquetes. Se agrega una *descripción* breve para mostrarla en la carta.

- **Ingrediente**: el cliente pidió saber cuánto queda de cada ingrediente y cuándo resurtir. Se modela como entidad, y no como un atributo de Producto, porque tiene datos propios (*unidad de medida*, *stock actual* y *stock mínimo*) y porque un mismo ingrediente, como el hielo, se usa en muchos productos: como atributo se repetiría en cada uno y no habría un solo stock del cual descontar.

- **Comanda**: es la entidad central del sistema y sustituye a la comanda de papel que hoy se entrega en caja. Registra *fecha*, *hora*, *cantidad*, *detalles* (las notas especiales que el cliente pidió conservar) y *estado*, con el cual se da seguimiento al ciclo pendiente, en preparación, lista, entregada o cancelada. Además guarda el *precio* al momento del pedido: como los precios cambian por temporada, conservarlo en la comanda evita que un cambio posterior en el catálogo altere los totales de eventos ya cobrados, se ah de aclarar que el sistema va a manejar cada producto de pedido como una comanda individual para mejorar el flujo de trabajo del sistema.

- **Pago**: el cliente acepta efectivo, transferencia y tarjeta, y necesita que quede registrado. La entidad guarda *monto*, *método de pago*, *fecha* y *hora*, lo que permite saber cómo y cuándo se liquidó cada cuenta. No se incluyen datos fiscales porque el cliente indicó que no requiere factura (CFDI).

- **Anticipo**: se separa del pago porque tiene un origen distinto: se recibe antes del evento y está ligado al evento completo, no a una cuenta de invitados. Mantenerlo aparte evita confundir los depósitos del organizador con los cobros realizados durante el evento.

- **Paquete de evento**: modela la comida preparada con antelación que el organizador contrata y que no se cobra aparte. Registra su *precio* y los *productos incluidos*, lo que establece lo presupuestado para el evento y lo distingue del consumo a la carta.

En todas las entidades se definió un identificador (código, número o identificador, subrayado en el diagrama) como clave primaria, de modo que cada registro pueda distinguirse de manera única aunque otros atributos, como el nombre o la fecha, se repitan.

3.5.2 Justificación de las relaciones

- **Usuario — Atiende — Mesa** (1:N): un mesero puede atender varias mesas. Refleja que, cuando hay varios meseros, a cada uno se le asigna un grupo de mesas, y que en eventos pequeños un solo mesero puede cubrirlas todas.

- **Mesa — Tiene — Cuenta** (1:N): una mesa puede tener varias cuentas, mientras que cada cuenta pertenece a una sola mesa. Es la relación que permite dividir el cobro por grupo de invitados, que es la forma de cobro más común en el salón.

- **Cuenta — Genera — Comanda** (1:N): una cuenta genera varias comandas y cada comanda pertenece a una sola cuenta. Así se representan las distintas rondas de pedidos que un mismo grupo realiza durante el evento, y el total de la cuenta puede obtenerse a partir de sus comandas.

- **Usuario — Atiende — Comanda** (M:N): en una comanda intervienen varios usuarios (el mesero que la levanta, el cajero que la recibe y la cocina o el bar que la preparan), y cada usuario participa en muchas comandas. Esta relación responde a la necesidad expresa del cliente de saber qué mesero hizo cada orden.

- **Comanda — Referencia — Producto** (M:1): cada comanda hace referencia a un producto del catálogo, y la cantidad y el precio pedidos quedan registrados en la propia comanda. Con ello se evita escribir a mano el nombre y el precio del producto en cada pedido.

- **Cuenta — Recibe — Pago** (1:N): una cuenta puede liquidarse con uno o varios pagos, por ejemplo cuando parte se paga en efectivo y parte con tarjeta; cada pago corresponde a una sola cuenta.

- **Evento — Recibe — Anticipo** (1:N): un evento puede recibir uno o varios anticipos antes de su fecha, y cada anticipo pertenece a un único evento.

- **Evento — Incluye — Paquete de evento** (M:N): un evento puede contratar uno o varios paquetes y un mismo paquete puede ofrecerse en muchos eventos, por lo que los paquetes se definen una sola vez y se reutilizan.

- **Producto — Receta — Ingrediente** (M:N): un producto lleva varios ingredientes y un ingrediente se usa en varios productos. La relación guarda como atributo la *cantidad* que lleva una porción, que no pertenece ni al producto ni al ingrediente sino a la pareja; con ella se descuenta el stock cada vez que se prepara una comanda.

- **Paquete de evento — Incluye — Producto** (M:N): un paquete agrupa varios productos y un producto puede formar parte de varios paquetes. Esta relación deja registrado qué productos están incluidos en cada paquete.

3.5.3 Cómo resuelve el modelo la problemática planteada

La problemática descrita en la introducción se origina en la gestión manual de las comandas y en un sistema anterior basado en unicamente papeles y tablas, sin relaciones bien definidas ni reglas de integridad. El modelo propuesto atiende estos problemas de la siguiente manera:

- **Pérdida y duplicación de pedidos entre meseros, cocina y caja**: la comanda deja de ser una hoja de papel y se convierte en un registro único, con un código propio, que todos los roles consultan. Su atributo *estado* permite a caja, cocina y bar saber en qué punto se encuentra cada pedido sin depender de mensajes verbales.

- **Falta de un registro histórico confiable**: cada comanda queda ligada a la cuenta que la generó, al producto que se pidió y a los usuarios que la atendieron, con fecha, hora y el precio vigente en ese momento. Esto permite reconstruir qué se sirvió, a quién y a qué costo.

- **Cuenta final lenta y propensa a errores**: al separar las cuentas por grupo de invitados y ligar a cada cuenta sus comandas y sus pagos, el total de cada grupo se obtiene directamente de los datos registrados, sin conciliar manualmente varias comandas en papel. Los anticipos, al estar ligados al evento, quedan registrados para descontarse del cobro final.

- **Control de responsabilidad del personal**: la relación entre usuario y comanda deja constancia de qué mesero levantó cada orden y quién más intervino en ella, lo que el cliente señaló como necesario.
- **Diferencia entre lo contratado y lo consumido a la carta**: los paquetes de evento y sus productos incluidos documentan lo que el organizador contrató con antelación, separado de las comandas a la carta que sí se cobran.
- **Deficiencias del sistema anterior**: a diferencia de los archivos ASCII del proyecto previo, cada entidad cuenta con una clave primaria y cada relación tiene una cardinalidad explícita. Esto permite, en la implementación en PostgreSQL, aplicar atributos fuertes y restricciones de integridad que impidan, por ejemplo, registrar un pago sin cuenta o una comanda que haga referencia a un producto inexistente.

En conjunto, el modelo traslada a una estructura formal las reglas de operación que el cliente describió en la entrevista, y constituye la base sobre la cual se implementará la base de datos del proyecto final.

Referencias

Aguilar Vera, R., Naal Jácome, A., Díaz Mendoza, J., & Gómez Gómez, O. (2023). NoSQL database modeling and management: A systematic literature review. *Revista Facultad de Ingeniería*, *32*(65), Artículo e16519. https://doi.org/10.19053/01211129.v32.n65.2023.16519

Bellinger, G., Castro, D., & Mills, A. (2004). *Data, information, knowledge, and wisdom*. Universidade Federal de Minas Gerais (UFMG). Consultado el 18 de septiembre de 2026, desde https://homepages.dcc.ufmg.br/~amendes/SistemasInformacaoTP/TextosBasicos/Data-Information-Knowledge.pdf

Chen, P. P.-S. (1976). The entity-relationship model—toward a unified view of data. *ACM Transactions on Database Systems*, *1*(1), 9-36. https://doi.org/10.1145/320434.320440

Date, C. J. (2001). *Introducción a los sistemas de bases de datos* (S. L. M. Ruíz Faudón, Trad.; 7.ª ed.). Pearson Educación.

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7.ª ed.). Pearson.

Machuca, L., & Braul, R. (2022). Metodologías más usadas en la seguridad de bases de datos: Una revisión de la literatura científica, 2016–2021. *Campus*, *27*(33), 121-130. https://doi.org/10.24265/campus.2022.v27n33.08

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2002). *Database system concepts: Instructor's manual* (4.ª ed.). McGraw-Hill Higher Education.

Taipalus, T. (2024). Database management system performance comparisons: A systematic literature review. *Journal of Systems and Software*, *208*, Artículo 111872. https://doi.org/10.1016/j.jss.2023.111872